

Concurrency Coding Assignment Writeup

Anna Wu

21 November 2025

1 Counting Semaphore

I believe my semaphore implementation is on its way to working. I've implemented a `cas` function for testing, and in the `cas` function I used a `pthread` mutex lock to make it atomic, and when that is the case the testing output looked reasonable to me (see below). I do think there might be an issue with the `sem_acquire` function, being that outside of any synchronization construct I check the value `sem->cas_loc` which represents the number of open spots in the critical section. My thinking here is that even if multiple threads get the signal that there is an open spot, because the `cas` function is atomic, only one of them should be able to get in and change it before they are let into the critical section. So one of them will do that, and then the rest will try to go in and the equality check will fail, so they will go back to spinning. This might not be a fully correct implementation but I think it has the basic ideas okay. For submission, I removed all of the mutex locks that I passed through the functions to the `cas` function, which I am assuming is what I am meant to do because I am allowed to assume the `cas` function is atomic. So if you run my code as is with my `cas`, it won't work because threads can change the same `cas_loc` value at the same time, but that shouldn't happen if the `cas` is atomic.

1.1 Testing

```
thread 0: acquiring
thread 0: inside critical section!!
open spots: 2
thread 1: acquiring
thread 1: inside critical section!!
open spots: 1
thread 2: acquiring
thread 2: inside critical section!!
open spots: 0
thread 4: acquiring
thread 3: acquiring
thread 8: acquiring
thread 6: acquiring
thread 7: acquiring
thread 9: acquiring
thread 5: acquiring
thread 0: releasing
open spots: 0
thread 9: inside critical section!!
open spots: 0
thread 2: releasing
open spots: 0
thread 7: inside critical section!!
open spots: 0
thread 1: releasing
```

```

open spots: 0
thread 8: inside critical section!!
open spots: 0
thread 9: releasing
open spots: 0
thread 7: releasing
open spots: 1
thread 8: releasing
open spots: 0
thread 3: inside critical section!!
open spots: 0
thread 5: inside critical section!!
open spots: 0
thread 4: inside critical section!!
open spots: 0
thread 3: releasing
open spots: 0
thread 6: inside critical section!!
open spots: 0
thread 5: releasing
open spots: 0
thread 4: releasing
open spots: 0
thread 6: releasing
open spots: 2

```

In `test.c` you can see what test I ran for this, which was basically 10 threads do a sequence where they acquire, the critical section is waiting for some seconds, and then they release, and I set the semaphore count to 3 threads at a time. You can see from the output that the first three threads successfully acquire, the rest are waiting in acquire, and then for the rest of the sequence a new thread never enters the section until a different thread releases. That enforced pattern is what makes me think the code is mostly correct. The repeated "open spots: 0" is from the tests not accessing that value exclusively either.

2 Dispatching Tasks

I use a lock here as my synchronization construct. Each thread checks the shared index pointer to the index in the list they should be working on, and that check as well as the increment is behind a lock, so only one thread can move on to its next task at a time until the index goes beyond the length of the list. So, threads start the next task as soon as they complete the one they were working on, and that might not be the next index sequentially in the array if another thread already took it. I use `pthread_join` at the end (and in testing functions too) and admittedly I couldn't figure out entirely what it actually does from the documentation I was reading, I just know that it works when I include it and doesn't work if I don't.

2.1 Testing

```

inputs:
0 1 2 3 4 5 6 7 8 9
thread 0x16d44f000 doing index 0
thread 0x16d4db000 doing index 1
thread 0x16d44f000 doing index 2
thread 0x16d4db000 doing index 3
thread 0x16d44f000 doing index 4
thread 0x16d44f000 doing index 5
thread 0x16d44f000 doing index 6

```

```

thread 0x16d44f000 doing index 7
thread 0x16d4db000 doing index 8
thread 0x16d4db000 doing index 9
thread 0x16d44f000 done because no more tasks
thread 0x16d4db000 done because no more tasks
outputs:
0 2 4 6 8 10 12 14 16 18

```

In `test.c` I implemented the `times_two` example, where I give it an array of integers from 1-10 and it should double them. I initialize two threads in my testing function and also give the `times_two` function a random wait each call because the first time I tested without it I couldn't tell if it actually was synchronized because the first thread could do almost all of them so quickly. With the waiting, I get the above output, which makes me think it is working because the threads trade off which ones they are doing and don't overlap. I didn't test it with more threads though so it is possible something is off about it, I wouldn't be completely sure.

3 Scheduling Resources

Answer to 3a:

To prevent deadlock in a system with these three resources, we have to enforce that a thread is not allowed to hold onto a resource while waiting for another one to become available. So on a request where the resource isn't available, the thread must release all the resources it already has. It must then try to acquire them all again from the start. That essentially means a thread can only wait for a resource if the number of resources it is holding is 0.

This should prevent deadlock, because resources will go back into the available pool for other threads when a thread is unsuccessful at acquiring all of them in a row. I think this still would allow livelock, but in the scope of this question we're just trying to prevent deadlock and the likelihood of livelock is low.

3.1 Testing

```

p1: using network and hard drive
p1 releasing network
p1 releasing hard drive
p1 complete
p3: using screen and network
p3 complete
p2: using hard drive and screen
p2 complete

```

I initialized three different processes to run on three different threads to simulate the example problem for this one, where they have those circular requests for the three resources. In my controller, I lock when a thread starts requesting a resource and only unlock if they are waiting or successfully got it. When I run the threads I get the above output, which makes me think it is working relatively well because of the order they complete them in. Because they all want a same resource as each other thread, only one thread can run at a time. So p1 gets the resources first, and upon completion it releases the network, which is the only other thing p3 tries to get. The screen is free the whole time because p1 doesn't use it, so when the network is freed p3 gets it quickly because it doesn't have to wait for the screen also, whereas p2 is using the hard drive which is released second, so it is too late to the screen that p3 has already picked up. So, it makes sense that p3 runs next and then upon completion p2 is the only one left requesting resources.

I did not test this on threads wanting a number of resources that isn't 2 (i.e. 1 or 3), and I also didn't check for processes that want to use the exact same two resources so I'm not certain if maybe it livelocks more often in one of those cases or something about this doesn't work for that. But I do feel pretty solidly that this solves the originally proposed issue of circular requests.